John Ian R. So

johnianrso@berkeley.edu johnianrso johnrso (925) 336-7553

Education

University of California, Berkeley **GPA: 4.00 / 4.00**

B.S. Electrical Engineering and Computer Science, May 2023

Courses Completed: Structure & Interpretation of Computer Programs, Data Structures, Machine Structures, Discrete Mathematics & Probability Theory, Designing Information Devices & Systems I, Self-Driving Cars, Multivariable Calculus, Physics for Scientists & Engineers.

Courses in Progress: Efficient Algorithms & Intractable Problems, Designing Information Devices & Systems II.

Technical Experience

Robotics Intern, PhaseSpace Motion Capture *May 2020-August 2020*

- Researched high contact surface sanitization methods, including UV-C and hypochlorous acid (HOCl), as a response to the COVID-19 pandemic.
- Prototyped software, firmware, and hardware using an ESP-32 controller and Firebase for a web interfaced, closed loop control system for a HOCl generator and dispenser system.
- Wrote and submitted a project proposal to the Department of Homeland Security SVIP Program for $200,000 for the HOCl system. Results pending.
- Implemented robot system architecture in ROS Melodic. Modelled a robot frame using URDF and simulated sensors and actuators in Gazebo. Designed a navigation software stack based on Real-Time Appearance-Based Mapping, Adaptive Monte Carlo Localization, and fusion of visual and IMU odometry.

Lead Intern, PhaseSpace Motion Capture *May 2018-August 2018*

- Led a team of four in building a robotic arm driven by an ESP-32 controller and PhaseSpace's proprietary motion capture system.
- Created a control system in Arduino to translate 3D vector space into arm joint angles. Developed software in Java to translate motion capture data into scaled vector representations and feed the data to the arm's control system.
- Machined the arm from scratch using parts modelled in Autodesk Fusion 360.

EECS Engineer, Formula Electric at Berkeley *January 2020-Present*

- Responsible engineer for the tractive system indicator and driver interface subsystems of an electric vehicle for the Formula SAE Competition EV division.
- Completed PCB designs for the tractive system indicator and driver interface in Altium Designer. Currently creating firmware for STM controllers in Mbed OS.

Teaching Experience

Course Staff, CS 61A *August 2020-Present*

- Paid course tutor for CS 61A: *Structure & Interpretation of Computer Programs*, Berkeley's largest computer science course with over 1500 students.
- Responsible for leading tutorials for group discussions four times per week and holding office hours twice per week.

Mentor, Computer Science Mentors *January 2020-Present*

- Leading weekly tutoring sessions for students in CS 61A, covering functional, object-oriented, and declarative programming in Python, Scheme, and SQL.

Skills

Languages: Java, C, C++, JavaScript, Python, Ruby, RISC-V, SQL, HTML/CSS.
Packages/Frameworks: ReactJS, Flask, Android Studio, OpenMP.
Robotics: ROS, OpenCV, Keras, TensorFlow, Arduino, ESP.
Tools: Git, Unix Shell, IntelliJ.
Circuit/PCB Design: Altium Designer, Multisim, Autodesk Eagle.

Projects

Mask Vision *August 2020-Present*

- Researched and implemented CNNs using Keras for detection of masks in images.
- Automated preparation of data sets using OpenCV and Keras. Currently looking into transfer learning using the YOLO architecture for real-time recognition of proper and improper usage of masks.

Word Hunt Solver *May 2020*

- Built a model-view-controller program for Word Hunt, a game where players swipe words on a 4x4 board to earn points.
- Implemented a solver that finds all possible words on the board within one second using DFS graph traversal.
- Currently deploying as a web application using Heroku, Flask, and ReactJS.

Mixer, CalHacks 6.0 *October 2019*

- Wrote a program using Spotify REST APIs that generates song recommendations based on seed songs and genres by scraping public playlists.
- Designed an algorithm to find similar songs and uploaded the selected songs to a Spotify playlist. Uploaded results to Firebase for future use.

Self-Driving Car Simulation *December 2019*

- Modelled and implemented self-driving car subsystems such as acceleration control, route planning, and steering control in Python using the Duckietown environment, OpenCV, and TensorFlow.
- Created models for linear regression and neural networks. Used PID, A*, and cost maps to plan and run routes.

NumC *July 2020-August 2020*

- Implemented a matrix library in C and a Python-C interface for use in Python. Supports element-wise addition, subtraction, negation, absolute value, and matrix multiplication/exponentiation.
- Optimized matrix operations using Intel SSE Intrinsics, OpenMP, loop unrolling, and cache blocking. Achieved a ~2400x speedup compared to unoptimized benchmarks.

Interests

Robotic perception & manipulation, autonomous vehicles, parallel computing, game theory, small scale computing